

VERIQTA



VERIQTA

VERIQTA

VERIQTA

VERIQTA

CI/CD PRODUCTION PROJECTS

VERIQTA

VERIQTA

VERIQTA

BUILD • TEST • SECURE • DEPLOY • OPERATE

VERIQTA



CODE



BUILD



TEST &
SECURE



PACKAGE



DEPLOY



OPERATE

VERIQTA

VERIQTA



VERIQTA

VERIQTA



REAL PROJECTS



PRACTICAL
PIPELINES



SECURITY
FIRST



KUBERNETES
DEPLOYMENTS



PRODUCTION
READY

From Commit to Production, The Right Way

VERIQTA

VERIQTA

VERIQTA

PROJECT ARCHITECTURE, FROM COMMIT TO PRODUCTION



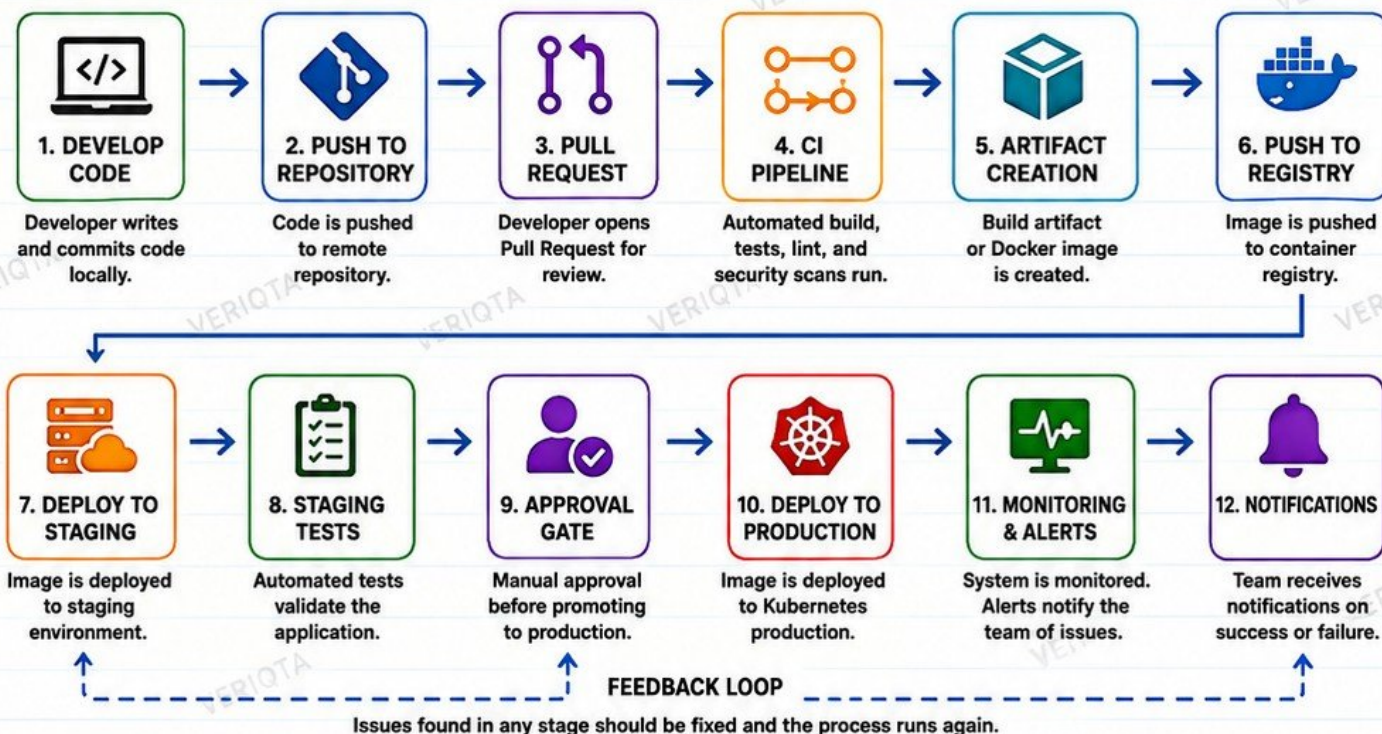
WHAT WE ARE BUILDING

We will build a complete CI/CD system that automatically takes code from development all the way to production safely, reliably, and repeatably.

KEY COMPONENTS

- Application Repository
- Developer Workflow
- Pull Request
- CI Pipeline
- Artifact Creation
- Container Registry
- Staging Environment
- Production Environment
- Kubernetes Deployment
- Monitoring & Notifications

COMPLETE DELIVERY-FLOW DIAGRAM



DEVOPS CONNECTION

This pipeline shows how one code change flows through the entire system. Everyone works together:



DEVELOPER

Writes and commits code.



DEVOPS ENGINEER

Builds and maintains the pipeline.



TEST ENGINEER

Creates tests and validates quality.



SECURITY ENGINEER

Ensures the system is safe and compliant.



SRE / OPS ENGINEER

Deploys, monitors, and keeps the system reliable.



REMEMBER: The goal is not just to deploy, but to deliver value safely and consistently.



VeriQTA
YouTube



veriqta
GitHub



veriqta
LinkedIn



veriqta
X



veriqta
Instagram



veriqta
Telegram

PROJECT 1, BUILD YOUR FIRST CI PIPELINE



GOAL

Create a simple CI pipeline that builds a small application every time code is pushed to the repository.



WHAT YOU WILL LEARN

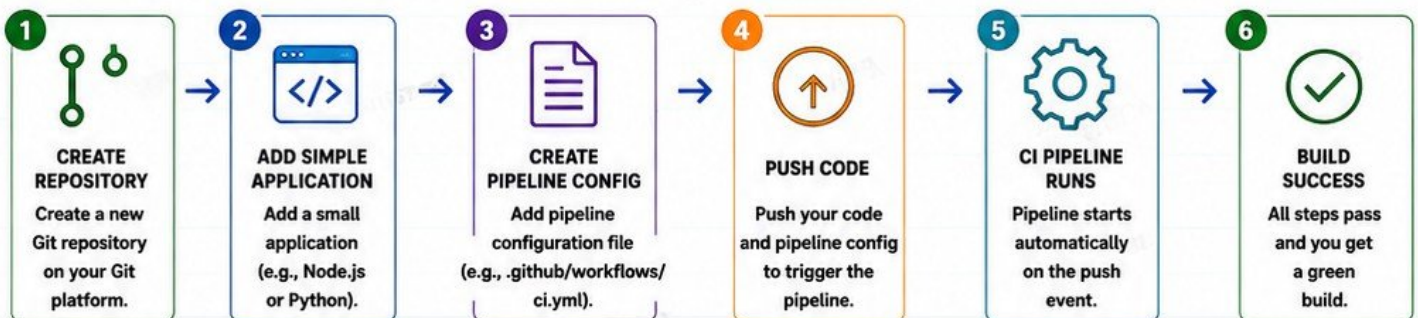
- How a CI pipeline is triggered
- What a pipeline, job, and step are
- How the build process works
- How to read pipeline logs
- How to find and fix a failure



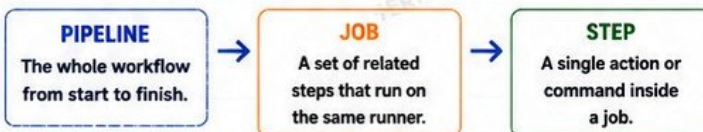
PRACTICE LAB

Make your first successful pipeline run from commit to green build.

STEP-BY-STEP WORKFLOW



TYPICAL CI PIPELINE STRUCTURE



EXAMPLE PIPELINE CONFIGURATION (GITHUB ACTIONS)

```
1 name: CI Pipeline
2
3 on:
4   push:
5     branches: [ main ]
6
7 jobs:
8   build:
9     runs-on: ubuntu-latest
10    steps:
11      - name: Checkout source code
12        uses: actions/checkout@v4
13      - name: Install dependencies
14        run: npm install
15      - name: Run build
16        run: npm run build
```

WHAT EACH STEP DOES

- **Checkout source code:** Pulls the latest code from the repository.
- **Install dependencies:** Installs required packages.
- **Run build:** Executes the build command to compile the application.

READING EXECUTION LOGS

```
✓ Set up job 2s
✓ Checkout source code 3s
✓ Install dependencies 18s
✓ Run build 6s
✓ Post job cleanup 1s
```

Pipeline completed successfully! 🎉

TIPS

- ✓ Click on any step to see detailed logs.
- ✓ Look for **✗** errors, warnings, and stack traces.
- ✓ The first error is usually the root cause.



INTENTIONALLY BREAK THE BUILD

- 1 Change your code to introduce an error. (e.g., syntax error or failing test)
- 2 Push the change to the repository.
- 3 Observe the pipeline fail.

WHY WE DO THIS

Engineers must be comfortable seeing failures. Pipelines fail often in real projects.



FIND AND REPAIR THE FAILURE

- 1 Read the logs carefully.
- 2 Identify the failing step.
- 3 Understand the error message.
- 4 Fix the code or configuration.
- 5 Push the fix and verify the pipeline.



PRACTICE LAB

Complete the full cycle:

- ✓ Create repository
- ✓ Add application
- ✓ Create pipeline config
- ✓ Push code
- ✓ Get a green build

You just built your first CI pipeline! ❤️



Veriqta
YouTube



veriqta
GitHub



veriqta
LinkedIn



veriqta
X



veriqta
Instagram



veriqta
Telegram

PROJECT 2, PULL REQUEST VALIDATION



WHY VALIDATE BEFORE MERGING?

- Protects the main branch from broken code
- Catches issues early, before they reach users
- Ensures code quality, security, and stability
- Builds confidence in every deployment

FEATURE BRANCH WORKFLOW



1. CREATE
FEATURE
BRANCH



2. MAKE
CHANGES &
COMMIT



3. PUSH
BRANCH TO
REMOTE



4. OPEN
PULL
REQUEST



5. MERGE TO
MAIN AFTER
VALIDATION

PULL REQUEST VALIDATION FLOW



1. OPEN PULL
REQUEST

Developer opens a
PR from feature
branch to main.



2. TRIGGER CI
AUTOMATICALLY

CI pipeline starts
automatically on
the PR.



3. BUILD
VALIDATION

Code is compiled
and dependencies
are installed.



4. UNIT
TESTS

Automated tests
run to verify code
works as expected.



5. LINTING
& CODE CHECKS

Linting and static
analysis ensure code
quality and style.



6. REQUIRED
STATUS CHECKS

All required checks
must pass before
merge is allowed.

STATUS CHECK EXAMPLE (GITHUB)



All checks have passed

3 successful checks

[Hide all checks](#)



build / build (ubuntu-latest)

Successful



test / unit-tests (ubuntu-latest)

Successful



lint / eslint (ubuntu-latest)

Successful



Merging blocked

This pull request is not mergeable.

1 failing check

[Show details](#)



BLOCK A FAILING PULL REQUEST

- If any required check fails, the PR cannot be merged.
- This prevents broken code from reaching main.
- Fix the issue in your branch.
- Push the changes.
- The pipeline runs again automatically.
- Merge becomes available once all checks pass.

TYPICAL CHECKS IN A PR PIPELINE



Build



Unit Tests



Linting



Security Scan



Dependency Check



Code Quality Scan



Integration Tests



License Check



BEGINNER MISTAKE

Merging because "it works on my machine"

- Your machine is not the production environment.
- Missing tests may hide bugs.
- Your environment may have different versions.
- Always trust the pipeline, not your local setup.



PRACTICE LAB

- 1 Create a feature branch.
- 2 Make a code change.
- 3 Open a pull request.
- 4 Watch the pipeline run.
- 5 Intentionally break a test.
- 6 Observe the failing status.
- 7 Fix the issue and push again.
- 8 Merge after all checks pass.



DEVOPS CONNECTION

PR validation is the gate that ensures only high-quality, tested, and secure code reaches production.

It is the first line of defense in a reliable delivery pipeline.



CODE



PR VALIDATION



BUILD



DEPLOY

PROJECT 3, AUTOMATED TESTING PIPELINE



GOAL

Build automated tests into the pipeline to catch bugs early and prevent broken code from reaching users.



WHAT YOU WILL LEARN

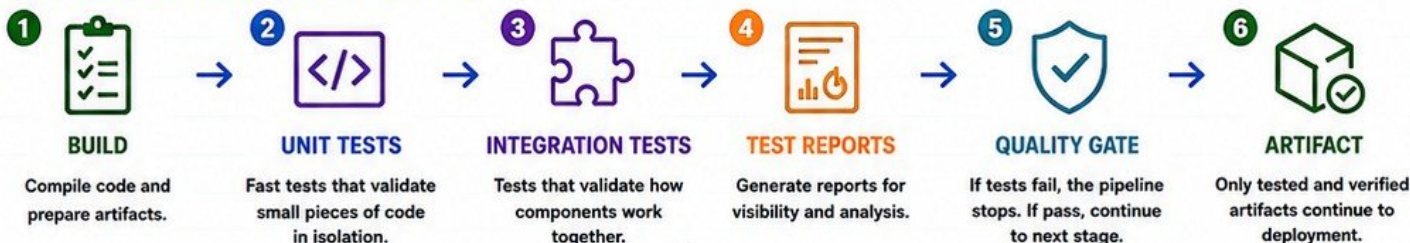
- Unit tests and integration tests
- Test stages and dependencies
- Test reports and exit codes
- Why failed tests stop pipelines
- Parallel vs sequential testing
- How to debug failing tests



PRODUCTION AWARENESS

Never treat a green build as proof that the application is healthy. Tests prove parts of the system, not the entire system.

AUTOMATED TESTING PIPELINE STAGES



TEST TYPES

TEST TYPE	PURPOSE	EXAMPLE
UNIT TESTS	Validate individual functions or classes.	Jest, PyTest, JUnit
INTEGRATION TESTS	Validate interactions between components.	API tests, DB tests, Service tests
END-TO-END TESTS	Validate the whole application flow.	Cypress, Playwright, Selenium

EXIT CODES IN TESTING

EXIT CODE	MEANING	PIPELINE ACTION
0	All tests passed	Pipeline continues
1	One or more tests failed	Pipeline stops
>1	Test framework error	Pipeline stops



WHY FAILED TESTS STOP PIPELINES

- Prevents bad code from reaching users.
- Avoids broken builds in later stages.
- Saves time and resources.
- Maintains code quality and trust.

PARALLEL VS SEQUENTIAL TESTING

SEQUENTIAL

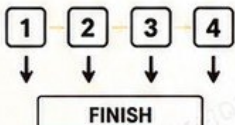
Tests run one after another.



- Simple to set up
- Slower for large test suites

PARALLEL

Tests run at the same time in different workers.



- Faster execution
- Requires isolated test environments

INTENTIONALLY INTRODUCE A FAILING TEST

- 1 Add a test that will fail.
- 2 Push the change to trigger the pipeline.
- 3 Observe the pipeline fail in the test stage.



TRACE THE FAILURE FROM PIPELINE LOGS

- 1 Open the pipeline run.
- 2 Go to the failed test stage.
- 3 Click the failed job.
- 4 Read the logs carefully.
- 5 Identify the failing test and error message.

```
Running tests...
Test Suite: user.test.js
✓ should login with valid user
✗ should reject invalid password
Error: Expected 200 but got 401
at user.test.js:45:12
Test Suites: 1 failed, 1 passed
Tests: 1 failed, 5 passed
Time: 2.345 s
error Command failed with exit code 1
```



FIX AND VERIFY

- 1 Fix the failing test or the code.
- 2 Push the change.
- 3 Pipeline runs again.
- 4 All tests pass.
- 5 Pipeline continues.



TEST REPORTS

- Provide visibility into test results.
- Show passed, failed, skipped tests.
- Help identify flaky or unstable tests.
- Store reports as pipeline artifacts.
- Share with the team for analysis.



TEST DEPENDENCIES

- Ensure databases, services, and mocks are available.
- Use containers or service dependencies in the pipeline.
- Isolate test environments.
- Clean up after tests.



BEGINNER MISTAKE

Merging because "it works on my machine"

- ✗ Your environment is different from production.
- ✗ Manual testing does not scale.
- ✗ Missing tests may hide bugs.
- ✗ Always trust the pipeline, not your local setup.



PROJECT 4, BUILD AND VERSION AN ARTIFACT



WHAT IS A BUILD ARTIFACT?

A build artifact is the output produced by the build process that is ready to be tested, stored, and deployed.



BUILD ONCE, DEPLOY MANY

Build once in CI and use the same artifact across all environments. This ensures consistency and traceability.



WHY IT MATTERS

- Guarantees consistency
- Improves reliability
- Enables rollback
- Creates audit trail

ARTIFACT NAMING EXAMPLES

TYPE	EXAMPLE	DESCRIPTION
Simple Version	app-1.0.0.jar	Basic version number
Version + Build Number	app-1.0.0-build.45.jar	Version with build identifier
Version + Date	app-1.0.0-20240520.jar	Version with build date
Commit SHA	app-1.0.0-a1b2c3d.jar	Version with short commit SHA
Full Metadata	app-1.0.0-a1b2c3d-20240520.1345.jar	Version + SHA + date + time

SEMANTIC VERSIONING INTRODUCTION



EXAMPLES

- 1.0.0 → Initial release
- 2.0.0 → Breaking changes
- 2.1.0 → New features
- 2.1.1 → Bug fixes

COMMIT SHA AS A VERSION

Commit SHA Example

a1b2c3d4e5f6g7h8i9j0

- Unique identifier for the exact code
- Short SHAs are commonly used (7-12 chars)
- Ensures traceability from artifact to source
- Combine SHA with version for clarity

Example: app-1.2.3-a1b2c3d.jar

BUILD METADATA

Version : 1.2.3
Commit SHA : a1b2c3d
Build Number : 45
Build Date : 2024-05-20
Build Time : 13:45:12 UTC
Built By : CI Pipeline
Branch : main
CI Run ID : 7890



- Metadata helps answer:
- What exactly was built?
 - When was it built?
 - From which code?
 - By which pipeline run?

STORE ARTIFACTS



- Store in artifact repository (e.g., Nexus, Artifactory, S3, or CI artifact storage)
- Use immutable storage
- Organize by name and version
- Keep retention policy

RETRIEVE ARTIFACTS



- Fetch the exact artifact needed for deployment
- Never rebuild in later stages
- Use version or checksum to ensure integrity
- Verify integrity before use

IMMUTABLE ARTIFACTS



- Once published, artifacts must never be changed
- Immutability ensures trust and traceability
- Prevents "it works on my machine" problems

WHY REBUILDING IS DANGEROUS



- Code or dependencies may change over time
- Builds may produce different results
- Hidden environment drift
- Cannot guarantee the same binary
- Breaks traceability and auditability

DELIVERY FLOW: SOURCE → BUILD → ARTIFACT → ENVIRONMENTS



SOURCE CODE

Developer writes code and commits to repo.



BUILD

CI builds the code and runs all necessary tasks.



ARTIFACT

A versioned, immutable artifact is created.



ARTIFACT REPOSITORY

Artifact is stored securely with metadata and version.



ENVIRONMENTS

Same artifact is deployed across environments.

STAGING

PRODUCTION

DR / BACKUP



JUNIOR ENGINEER TIP

Know exactly what binary or package reached production. Traceability is your best friend during incidents and audits.



KEY TAKEAWAYS

- Build once in CI
- Version everything
- Store immutably
- Deploy the same artifact everywhere

6. PROJECT 5, DOCKERIZE THE APPLICATION



WHAT IS DOCKER?

Docker packages the application and all its dependencies into a lightweight, portable container so it runs the same everywhere.

WHY CONTAINERIZE?

- ✓ Consistent environments
- ✓ Works on any server or cloud
- ✓ Faster deployments
- ✓ Easy scaling
- ✓ Simplifies CI/CD



GOAL

Turn our application into a Docker image that can be built, tested, and deployed reliably.

DOCKERIZATION STEPS



1. 1 DOCKERFILE EXAMPLE (NODE.JS APP)

```
Dockerfile
1 # 1. Choose base image
2 FROM node:20-alpine
3
4 # 2. Set working directory
5 WORKDIR /app
6
7 # 3. Copy dependency files
8 COPY package*.json ./
9
10 # 4. Install dependencies
11 RUN npm install --production
12
13 # 5. Copy application files
14 COPY . .
15
16 # 6. Expose application port
17 EXPOSE 3000
18
19 # 7. Set startup command
20 CMD ["node", "server.js"]
```

LINE-BY-LINE

- ✓ FROM: Base image
- ✓ WORKDIR: Working directory
- ✓ COPY: Copy files
- ✓ RUN: Install packages
- ✓ EXPOSE: Application port
- ✓ CMD: Startup command

2. BUILD AND RUN THE IMAGE

```
$ docker build -t myapp:1.0.0 . # Build the image
$ docker images # List images

$ docker run -d -p 3000:3000 \
  --name myapp-container \
  myapp:1.0.0 # Run container
```

ACCESS YOUR APP

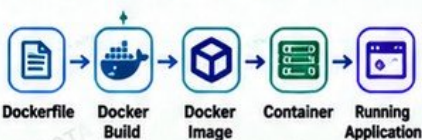
<http://localhost:3000>

✓ App is running!

3. IMAGE TAGGING

TAG	PURPOSE
myapp:1.0.0	Version tag
myapp:latest	Latest build (avoid for production)
myapp:a1b2c3d	Tag with commit SHA
myapp:2024-05-20	Tag with build date

CONTAINER LIFECYCLE DIAGRAM



4. INSPECT CONTAINER LOGS

```
$ docker logs myapp-container # View container logs
$ docker ps # Check container status
$ docker inspect myapp-container # Inspect full details
```



BEGINNER MISTAKE

Application works on my machine but fails inside the container

- ✗ Missing environment variables
- ✗ Different Node/Python version
- ✗ Missing system dependencies
- ✗ Wrong working directory
- ✗ Forgetting to expose the port



PRODUCTION TIPS

- ✓ Use small base images (alpine, slim)
- ✓ Use .dockerignore
- ✓ Tag images with commit SHA
- ✓ Keep images immutable
- ✓ Test the container before pushing
- ✓ Treat the container as production code



COMMON FAILURES & FIXES

PROBLEM	CAUSE	FIX
App starts then exits	Wrong CMD	Check startup command
Port not accessible	Wrong EXPOSE/port	Verify port mapping
Module not found	Missing dependency	Install dependencies
Permission error	Non-root user issue	Use correct user/permissions
Works locally, fails in container	Environment difference	Match runtime env and check logs



JUNIOR ENGINEER TIP

A good Dockerfile means the exact same application runs on your laptop, CI, staging, and production — with no "it works on my machine" issues.



7. PROJECT 6, BUILD DOCKER IMAGES IN CI

We turn our application into a Docker image inside the CI pipeline.

1. ADD DOCKER BUILD

Add Docker steps after tests pass.

- Build the image
- Tag with commit SHA
- Tag with version (on release)
- Push to registry



2. EXAMPLE CI STEP (GITHUB ACTIONS)

```
- name: Build and Push Docker Image
  uses: docker/build-push-action@v5
  with:
    context: .
    file: ./Dockerfile
    push: true
    tags: |
      registry.example.com/myapp:${{ github.sha }}
      registry.example.com/myapp:${{ github.ref_name }}
  labels: |
    org.opencontainers.image.revision=${{ github.sha }}
    org.opencontainers.image.source=https://github.com/org/repo
    org.opencontainers.image.created=${{ github.run_id }}
```

3. IMAGE NAMING CONVENTION

Use a clear and consistent format.

`registry.example.com/<team>/<app>:<tag>`

Examples:

- `registry.example.com/team/api:1.2.3`
- `registry.example.com/team/api:abc1234`
- `registry.example.com/team/api:latest`

5. TAGGING STRATEGY

- ✓ **COMMIT SHA** (Immutable)
Unique for every code change.
Perfect for traceability.
- ✓ **RELEASE VERSION** (Immutable)
Use semantic versioning
for human readability.
Tag on release branches.
- ✓ **LATEST** (Mutable)
Use only for non-production.
Never deploy :latest in
production.

6. BUILD CONTEXT

The build context is the set of files Docker can see during build.

✓ GOOD

- Use `.dockerignore`
- Include only what is needed
- Smaller context = faster builds
- Better security

Example `.dockerignore`

```
node_modules
.git
*.log
.env
```

✗ BAD

- Large context
- Includes secrets
- Includes unnecessary files
- Slower builds
- Higher risk

4. AVOID RELYING ONLY ON :LATEST

- Mutable tags cause surprises.
- Always use immutable tags (SHA or version) in production.
- Use :latest only for local dev or non-production testing.

7. DOCKER LAYER CACHING

Docker builds in layers.
Use caching to speed builds.

- Keep stable layers first
- Separate dependency install from code copy
- Cache dependencies
- Avoid invalidating cache unnecessarily



8. FAILED IMAGE BUILDS

Common causes:

- Dockerfile syntax error
- Missing files in context
- Dependency download failure
- Out of disk space
- Network issues
- Wrong base image
- Permission problems

What to do:

1. Read the exact error
2. Check the failed step logs
3. Reproduce locally if needed
4. Fix the root cause
5. Re-run the pipeline



9. VERIFY THE BUILT IMAGE

After build, verify:

- ✓ Image exists in registry
- ✓ Tags are correct
- ✓ Image runs locally
- ✓ Application starts
- ✓ Health check works
- ✓ Image size is reasonable
- ✓ No secrets in layers



10. PIPELINE FLOW DIAGRAM



CODE
COMMIT



GIT
REPOSITORY



CI PIPELINE
(BUILD & TEST)



DOCKER
BUILD



IMAGE
CREATED



PUSH TO
REGISTRY



READY FOR
DEPLOYMENT



PRODUCTION AWARENESS

- Use immutable tags (SHA or version)
- Keep build metadata (labels)

EVERY DEPLOYED IMAGE MUST BE TRACEABLE TO SOURCE CODE.

- Record who, what, when, and why
- Traceability = Accountability + Reliability

8. PROJECT 7, PUSH IMAGES TO A CONTAINER REGISTRY

Store and share container images securely so environments can pull exactly what was built.

1. WHAT A CONTAINER REGISTRY DOES



- Stores Docker images.
- Provides versioned, shareable images.
- Allows CI/CD, servers, and Kubernetes to pull images.
- Examples: Docker Hub, GHCR, ECR, GCR, ACR.

2. REGISTRY AUTHENTICATION



- Authenticate before pushing or pulling.
- Methods: Username/Password, Token, Personal Access Token, OIDC (recommended for cloud).

```
# Example login (Docker Hub)
echo $DOCKER_PASSWORD | docker login -u
$DOCKER_USERNAME --password-stdin
```

3. REPOSITORY NAMING



- Format: <registry>/<namespace>/<repo>
- Use your org/username as namespace.
 - Keep names lowercase.
 - Example:

```
registry.example.com/team/myapp
```

4. IMAGE TAGS



- Tags identify image versions.
- Good practice: multiple tags per build.
- Common tags:
 - git SHA (immutable)
 - semantic version (v1.2.3)
 - build number (optional)
 - latest (mutable, use carefully)

5. PUSH FROM CI



- After image build, push to registry.
- Tag first, then push.
- Example:

```
docker tag myapp:$GITHUB_SHA
registry.example.com/team/myapp:$GITHUB_SHA

docker push registry.example.com/team/myapp:$GITHUB_SHA
docker push registry.example.com/team/myapp:v1.2.3
```

6. PULL VERIFICATION



- Always verify image can be pulled.
- Test pull in CI or from a clean machine.
- Example:

```
docker pull registry.example.com/team/myapp:$GITHUB_SHA
docker images | grep myapp
```

7. CREDENTIALS AND SECRETS



- Store credentials as CI/CD secrets.
- Never hardcode in Dockerfile or scripts.
- Use least-privilege accounts.
- Rotate tokens regularly.
- Use cloud IAM roles / OIDC for better security.

8. PREVENT CREDENTIALS IN LOGS



- Use secret masking in CI.
- Use --password-stdin instead of --password.
- Never echo secrets.
- Avoid set -x when handling secrets.
- Mask tokens in logs and output.

9. IMMUTABLE TAGS



- Immutable tags prevent overwrites.
- Enforce repository settings to block tag mutation.
- Always reference immutable tags in deployments (SHA or version).

10. IMAGE RETENTION BASICS



- Registries can grow quickly.
- Keep what you need, delete what you don't.
- Retention strategies:
 - ✓ Keep last N versions
 - ✓ Keep last N days
 - ✓ Keep tags matching patterns
 - ✓ Never delete release tags
- Automate cleanup to control storage cost.

11. INTENTIONALLY BREAK REGISTRY AUTHENTICATION



How to break it:

- Use wrong username
- Use wrong password/token
- Expired token
- Remove permission
- Use wrong registry URL

What you will see in logs:

```
denied: requested access to the resource is denied
unauthorized: incorrect username or password
Error response from daemon: Get
"https://registry.example.com/v2/": unauthorized
```

12. TROUBLESHOOTING TIP

Separate authentication failures from network and permission failures.

SYMPTOM	LIKELY CAUSE	HOW TO CHECK	EXAMPLE ERROR	FIX
Unauthorized / Denied	Bad credentials or no permission	Check username/token, permissions	unauthorized: incorrect username or password	Fix credentials or grant permission
Network error	Cannot reach registry	Ping / curl registry URL	dial tcp: i/o timeout	Check network, firewall, proxy
TLS / SSL error	Certificate or TLS issue	Check TLS settings, cert validity	x509: certificate signed by unknown authority	Fix certs or CA config
Not found	Wrong repository name or tag	Check repository path and tag	manifest unknown	Use correct repo or tag
Forbidden	Authenticated but no access	Check permissions	denied: requested access to the resource is denied	Grant required role or policy

13. REGISTRY FLOW



CI PIPELINE
Build succeeds



DOCKER IMAGE
Image is created and tagged



PUSH TO REGISTRY
Image is pushed securely



PULL BY ENVIRONMENTS
Staging / Production pull the exact image



PRODUCTION AWARENESS

- ✓ Use immutable tags (SHA or version)
- ✓ Record who built and pushed the image

EVERY DEPLOYED IMAGE MUST BE TRACEABLE TO SOURCE CODE.

- ✓ Record when it was built
- ✓ This ensures accountability and rollback safety

9. PROJECT 8, ADD SECURITY SCANNING

Security is not a separate step. It is part of the pipeline.

1. WHY SECURITY BELONGS INSIDE CI/CD



- Catch issues early when they are cheaper to fix.
- Prevent vulnerable code and images from reaching production.
- Reduce risk of breaches, compliance violations, and downtime.
- Build a culture of secure by default.

2. TYPES OF SECURITY SCANNING



DEPENDENCY SCANNING

Finds vulnerable libraries and packages.



CONTAINER IMAGE SCANNING

Finds OS packages and vulnerabilities inside images.



SECRET SCANNING

Detects hardcoded secrets, tokens, keys, passwords.



SAST (STATIC APP SECURITY TEST)

Analyzes source code for security flaws.



DAST (OPTIONAL)

Tests a running application for vulnerabilities.

3. SEVERITY LEVELS

CRITICAL

Immediate risk. Likely to be exploited.

HIGH

High risk. Fix as soon as possible.

MEDIUM

Moderate risk. Fix in regular cycle.

LOW

Low risk. Monitor and plan to fix.

INFO

Informational. Not a direct risk.

4. SECURITY GATES



Define rules that must be met before the pipeline continues.

- ✓ No Critical vulnerabilities
- ✓ No High vulnerabilities (policy)
- ✓ No secrets detected
- ✓ SAST quality gate passed
- ✓ License policy passed

5. FAIL BUILDS ON DEFINED CONDITIONS



Example policy:

- Fail on Critical
- Fail on High
- Fail on secrets
- Fail on SAST Blocker/Critical

Do not allow unsafe code or images to move to the next stage.

6. FALSE POSITIVES



- Scanners are not perfect.
- Review findings carefully.
- Use allowlists and ignore files when appropriate.
- Never ignore real issues.

Understand the risk before dismissing a finding.

7. VULNERABILITY REMEDIATION



- Update vulnerable dependencies.
- Upgrade base images.
- Apply security patches.
- Remove unnecessary packages.
- Fix insecure code.
- Regenerate secrets.

Fix the root cause, not just the symptom.

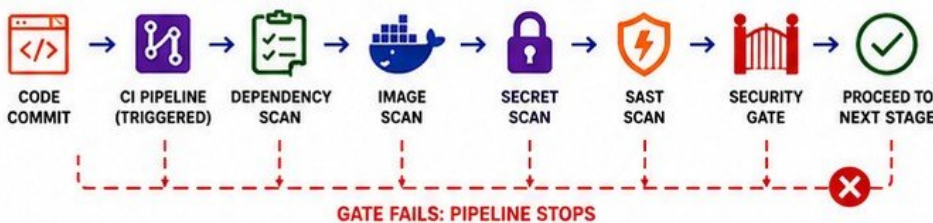
8. RESCAN AFTER REMEDIATION



- After fixes, run scans again.
- Verify vulnerabilities are gone or accepted.
- Ensure policies pass.
- Only then allow the pipeline to continue.

Trust but verify.

9. SECURITY SCANNING IN THE PIPELINE (EXAMPLE FLOW)



Security scans run automatically and results determine if the pipeline continues.

10. COMMON TOOLS (EXAMPLES)



- Dependency Scan (Snyk, Trivy, OWASP Dependency Check)
- Container Image Scan (Trivy, Clair, Gype)
- Secret Scan (GitLeaks, TruffleHog)
- SAST (SonarQube, Semgrep, Checkmarx)
- DAST (Optional) (OWASP ZAP, Burp Suite)



SECURITY REMINDER

A SUCCESSFUL BUILD DOES NOT MEAN A SAFE BUILD.
Only a secure build is a build that can reach production.



10. PROJECT 9, DEPLOY AUTOMATICALLY TO STAGING

Automatically deploy the exact tested artifact to the staging environment.

1. CREATE STAGING ENVIRONMENT



- Dedicated staging namespace.
- Isolated from production.
- Mirrors production as closely as possible.
- Used for validation before promotion.

2. DEPLOYMENT CONFIGURATION



- Define Deployment manifest.
- Define Service manifest.
- Define Ingress (if needed).
- Use declarative YAML.
- Store in version control.

3. ENVIRONMENT-SPECIFIC VALUES



- Use values for staging.
- Different URLs, replicas, resources, and configs.
- Example: values-staging.yaml
- Keep secrets separate.

4. DEPLOY THE EXACT TESTED IMAGE



- Use the image produced in CI.
- Never rebuild for staging.
- Tag with commit SHA or release version.
- Ensures traceability.

5. KUBERNETES DEPLOYMENT



- Defines desired state for Pods.
- Manages replicas.
- Handles rolling updates.
- Example: apps/v1 Deployment

6. KUBERNETES SERVICE



- Exposes Pods internally (ClusterIP) or externally (LoadBalancer).
- Stable endpoint for the application.

7. CONFIGMAP



- Store non-sensitive configuration.
- Separate config from the container image.
- Update without rebuilding the image.

8. SECRET



- Store sensitive data.
- Database credentials, API keys, etc.
- Base64 encoded in Kubernetes.
- Mount as environment variables or files.

9. WAIT FOR ROLLOUT



- Kubernetes rolls out the new Pods.
- Wait for rollout to complete.

```
kubectl rollout status deployment/myapp
```

10. CHECK POD STATUS



- Ensure Pods are Running.
- No CrashLoopBackOff.
- All containers ready.
- Commands:

```
kubectl get pods -n staging
kubectl get deployment myapp -n staging
```

11. INSPECT APPLICATION LOGS



- Verify application starts correctly.
- Check for errors or exceptions.
- Commands:

```
kubectl logs -f deployment/myapp -n staging
kubectl logs <pod-name> -n staging
```

12. VERIFY DEPLOYMENT



- Application endpoint responds.
- Health checks pass.
- Smoke test (basic checks).
- Confirm the correct version is running.

STAGING DEPLOYMENT FLOW (HIGH LEVEL)



DEVOPS CONNECTION

CI creates confidence by validating the code. CD moves that validated artifact into the staging environment automatically for further validation.

CI
BUILDS CONFIDENCE



CD
DELIVERS CONFIDENCE



REMEMBER

Staging is your safety net before production.
Trust but verify every deployment.

- ✓ Same artifact
- ✓ Same process

- ✓ Early feedback
- ✓ Lower risk

11. PROJECT 10, ADD SMOKE TESTS AFTER DEPLOYMENT



WHAT SMOKE TESTING VALIDATES

- The application is running and reachable
- Critical endpoints respond correctly
- Core functionality works at a basic level
- External dependencies are accessible
- The system is healthy from a user's perspective



WHY DEPLOYMENT SUCCESS IS NOT APPLICATION SUCCESS

- Kubernetes may report Pods as Ready even when the application is broken.
- Containers can start but the app may crash later.
- Dependencies like DB, cache, or external APIs may be failing.
- Only testing real endpoints proves the application is actually working.

HEALTH ENDPOINT



A simple endpoint the app exposes to show its basic health.

Example: `GET /health`

Expected Response:

`200 OK`

HTTP STATUS VALIDATION



Verify the endpoint returns the correct HTTP status.

Common Expectation:

Status: 200 - 299

Anything else indicates a problem.

BASIC API TEST



Call a real API that checks important functionality.

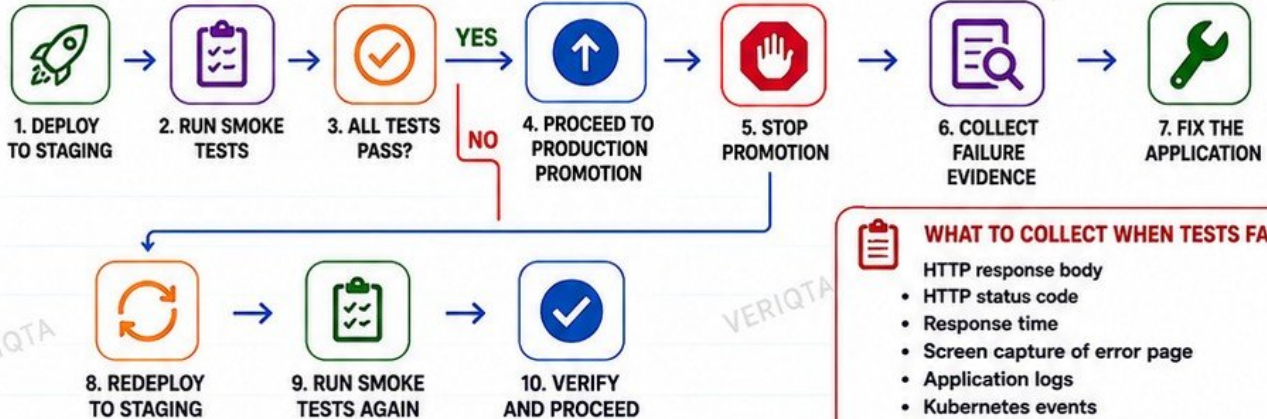
Example:

`GET /api/v1/status`

Expected Response:

`{"status": "ok"}`

SMOKE TEST WORKFLOW (RUN AFTER STAGING DEPLOYMENT)



WHAT TO COLLECT WHEN TESTS FAIL

- HTTP response body
- HTTP status code
- Response time
- Screen capture of error page
- Application logs
- Kubernetes events
- Timestamp and test details



REAL WORLD EXAMPLE

KUBERNETES REPORTS READY

- Pods are Running
- Readiness probe passes
- Service is reachable
- Load balancer shows healthy



APPLICATION WORKFLOW IS BROKEN

- User login fails due to DB connection issue
- External payment service returns 500
- API returns 200 but response data is incorrect
- Background job is crashing



LESSON

Never trust only infrastructure signals. Always validate the application like a real user after every deployment.



JUNIOR ENGINEER TIP

Smoke tests are cheap and fast. They prevent broken releases from reaching production.



COMMON MISTAKES

- Skipping smoke tests because "deployment succeeded"
- Only checking the home page
- Not validating critical user journeys



PRACTICE LAB

Add a smoke test stage to your pipeline that fails when the health endpoint returns 500.

12. PROJECT 11, ENVIRONMENT PROMOTION

ENVIRONMENTS



DEVELOPMENT

- Developers build and test
- Frequent changes



STAGING

- Mirrors production
- Used for full testing and validation



PRODUCTION

- Live environment
- Stable and protected

PROMOTION VS REBUILDING



PROMOTION (CORRECT)

- Build once and promote the same artifact through all environments.
- Ensures consistency and greatly reduces risk.
- Easy to trace and rollback.



REBUILDING (RISKY)

- Building a new image for each environment.
- Increases chance of "works on my machine" problems.
- Harder to trace and reproduce.

SAME ARTIFACT ACROSS ENVIRONMENTS



ARTIFACT
app:v1.2.3



DEV



STAGING



PRODUCTION

ONE BUILD, MANY ENVIRONMENTS



ENVIRONMENT CONFIGURATION

- Use external configuration (ConfigMaps, Environment Variables, Secrets).
- Keep environment-specific values outside the application package.
- Same artifact, different configuration.

DEV	→	api_url=dev.example.com
STAGING	→	api_url=staging.example.com
PRODUCTION	→	api_url=app.example.com



RELEASE CANDIDATE

- After testing in staging, the artifact becomes a Release Candidate (RC).
- The RC is promoted only after all validations pass.
- RC must be immutable.



APPROVAL GATES

- Human or automated approvals before production.
- Requires release evidence (tests, scans, verification).
- Prevents unverified changes from reaching users.



PRODUCTION PROTECTION

- Protected environments in CI/CD.
- Only authorized users can deploy.
- Audit trail for every release.

PROMOTION FLOW DIAGRAM



1. BUILD ONCE
Create artifact from source.



2. TEST
Run unit, integration, security tests.



3. STAGING
Deploy the same artifact to staging.



4. VERIFY
Run smoke tests and validations.



5. APPROVE
Manual or automated approval gate.



6. PRODUCTION
Promote artifact to production safely.

--- SAME ARTIFACT PROMOTED THROUGH ALL ENVIRONMENTS ---



BEGINNER MISTAKE

BUILDING A DIFFERENT IMAGE FOR EVERY ENVIRONMENT

DEV BUILD



app:dev-123

STAGING BUILD



app:staging-456

PROD BUILD



app:prod-789

WHY THIS IS BAD

- Inconsistent behavior between environments.
- Bugs are hard to reproduce.
- Increased risk of configuration drift.
- Longer time to diagnose issues.



REMEMBER

Promote the same tested artifact.

Control changes with configuration, not code.

Trust the pipeline, but verify the application.

13. PROJECT 12, PRODUCTION APPROVAL WORKFLOW



WHY SOME DEPLOYMENTS REQUIRE APPROVAL

- Production changes carry business risk.
- Some changes can impact customers, data, or revenue.
- Approval adds a deliberate human checkpoint.
- Provides time for review, validation, and communication.
- Required for compliance in many organizations.



AUTOMATED CHECKS BEFORE APPROVAL

- All tests pass (unit, integration, security, quality).
- Smoke tests in staging pass.
- Security scans pass.
- Performance and reliability checks pass.
- Infrastructure and policy checks pass.
- Change ticket / issue is linked.
- Release notes and documentation are attached.
- Only after all checks pass → request approval.

PRODUCTION APPROVAL WORKFLOW



1. BUILD & TEST

Code is built, tested, and scanned.



2. STAGING DEPLOYMENT

Artifact is deployed to staging.



3. AUTOMATED CHECKS

System runs smoke tests, security scans, and policy checks.



4. MANUAL APPROVAL

Human approver reviews evidence and approves the release.



5. PRODUCTION DEPLOYMENT

Artifact is deployed to production.



6. VERIFY & MONITOR

System health is verified and monitored.



WHO SHOULD APPROVE?

- Someone who understands the change and its impact.
- Not the person who made the change.
- Not the same person who deployed to staging.
- Usually: Team Lead, Tech Lead, Engineering Manager, or Change Manager.



SEPARATION OF DUTIES

- Developer writes code.
- CI system builds and tests.
- Operator deploys to staging.
- Approver reviews and approves.
- Another system deploys to production.
- No single person should control the entire release.



RELEASE EVIDENCE

- Test results and reports
- Security scan report
- Smoke test results
- Change summary
- Linked ticket / change request
- Release notes
- Rollback plan
- Evidence must be available before approval.



DEPLOYMENT WINDOW CONSIDERATIONS

- Deploy during agreed maintenance windows when risk is lower.
- Avoid business peak hours.
- Communicate maintenance windows in advance.
- Ensure rollback plan is ready before the window.



EMERGENCY DEPLOYMENT PATH

- Some issues require immediate action.
- Emergency path bypasses approval with audit logging.
- Used only for critical fixes.
- Requires post-deployment review.
- Always document why approval was bypassed.



AUDIT TRAIL

- All approval actions are logged.
- Who approved
- When it was approved
- What was approved
- Why it was approved
- Provides accountability and compliance.



ENTERPRISE ADVICE: APPROVAL SHOULD CONTROL RISK, NOT COMPENSATE FOR UNRELIABLE AUTOMATION

- If your pipeline is flaky, slow, or inconsistent, approval will not save you.
- Strong automated quality gates reduce the need for manual approvals.
- Approval should be reserved for managing business risk, not covering up weak engineering.
- Automation ensures quality. Approval ensures accountability.

14. PROJECT 13, KUBERNETES PRODUCTION DEPLOYMENT



PRODUCTION NAMESPACE

Isolate resources in a dedicated namespace.

```
namespace: app-prod
```



DEPLOYMENT MANIFEST

Define the desired state for your application.



REPLICA COUNT

Run multiple Pod instances for high availability.

```
replicas: 3
```



RESOURCE REQUESTS & LIMITS

Guarantee resources and prevent noisy neighbors.

```
requests: 250m / 256Mi
```

```
limits: 500m / 512Mi
```

EXAMPLE DEPLOYMENT MANIFEST (YAML)

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: app
5   namespace: app-prod
6   labels:
7     app: app
8 spec:
9   replicas: 3
10  strategy:
11    type: RollingUpdate
12    rollingUpdate:
13      maxUnavailable: 1
14      maxSurge: 1
15  selector:
16    matchLabels:
17      app: app
18  template:
19    metadata:
20      labels:
21        app: app
22    spec:
23      containers:
24        - name: app
25          image: registry.example.com/app:v1.2.3
26          imagePullPolicy: IfNotPresent
27          ports:
28            - containerPort: 8080
29          resources:
30            requests:
31              cpu: "250m"
32              memory: "256Mi"
33            limits:
34              cpu: "500m"
35              memory: "512Mi"
36          readinessProbe:
37            httpGet:
38              path: /health
39              port: 8080
40            initialDelaySeconds: 10
41            periodSeconds: 10
42            timeoutSeconds: 2
43            failureThreshold: 3
44          livenessProbe:
45            httpGet:
46              path: /live
47              port: 8080
48            initialDelaySeconds: 20
49            periodSeconds: 20
50            timeoutSeconds: 2
51            failureThreshold: 3
```

KEY CONFIGURATION EXPLAINED



REPLICA COUNT

Ensures high availability and zero downtime.



RESOURCE REQUESTS

Minimum resources guaranteed to the container.



RESOURCE LIMITS

Maximum resources the container can use.



READINESS PROBE

Determines when the Pod is ready to receive traffic.



LIVENESS PROBE

Detects dead or stuck containers and restarts them.



ROLLINGUPDATE STRATEGY

Updates Pods gradually without downtime.
maxUnavailable: 1, maxSurge: 1



DEPLOY IMMUTABLE IMAGE

Always deploy a versioned image, never use "latest".
Example: registry.example.com/app:v1.2.3



DEPLOY & CHECK ROLLOUT

```
# Apply the deployment
kubectl apply -f deployment.yaml
# Check rollout status
kubectl rollout status deployment/app -n app-prod
# Check rollout history
kubectl rollout history deployment/app -n app-prod
```



WHAT THIS CONFIRMS

- New Pods are created successfully
- Old Pods are terminated safely
- The deployment is progressing
- No unexpected errors



VERIFY PODS & SERVICES

- `kubectl get pods -n app-prod`
- `kubectl get svc -n app-prod`
- `kubectl get endpoints -n app-prod`

Ensure Pods are Running
Ensure Service has Endpoints
Ensure correct number of replicas



INSPECT EVENTS

- `kubectl get events -n app-prod --sort-by=.lastTimestamp`
- `kubectl describe deployment app -n app-prod`

Check for:

- Failed scheduling
- Image pull errors
- Probe failures
- Resource issues



COMMON QUICK CHECKS

- `kubectl get pods -n app-prod -o wide`
- `kubectl top pods -n app-prod`
- `kubectl logs -n app-prod <pod-name>`

Look for:

- Pod status
- CPU/Memory usage
- Application logs



PRODUCTION AWARENESS: A COMPLETED PIPELINE IS NOT THE SAME AS A SUCCESSFUL PRODUCTION RELEASE.

Always verify the actual behavior of the application in production.



Veriqta
YouTube



veriqta
GitHub



veriqta
LinkedIn



veriqta
X



veriqta
Instagram



veriqta
Telegram

15. PROJECT 14, DEPLOYMENT VERIFICATION

1 VERIFY ROLLOUT STATUS



- Ensure the rollout completed successfully.
- No updated or unavailable Pods.

```
kubectl rollout status
deploy/app -n app-prod
```

2 CHECK APPLICATION HEALTH



- Validate the app is healthy from inside the cluster.
- Use readiness endpoint or health check.

```
kubectl exec deploy/app -n app-prod
-- curl -f http://localhost:8080/health
```

3 TEST EXTERNAL ENDPOINT



- Test the real endpoint users will access.
- Validate response code and basic content.

```
curl -f -s -o /dev/null -w
"%{http_code}" https://
app.example.com/health
```

4 CHECK ERROR RATE



- Monitor 5xx errors.
- Compare with baseline before deployment.

Target: Keep error rate within acceptable threshold.

5 CHECK LATENCY



- Compare response time before and after deployment.
- Watch P95 and P99 latency.

Target: No significant latency regression.

6 CHECK POD RESTARTS



- Pods restarting indicates instability.
- Check restarts over the verification window.

```
kubectl get pods -n app-prod
-o wide
```

7 COMPARE PRE & POST DEPLOYMENT SIGNALS



- Compare metrics before and after deployment.
- Look for improvement or at least no degradation.

Use dashboards to compare relevant metrics.

8 DEFINE VERIFICATION WINDOW



- Give the system time to stabilize.
- Typical window: 10 to 30 minutes (depends on app).

Wait, observe, and collect enough data.

✓ RELEASE SUCCESS CRITERIA

A deployment is considered successful only when ALL of the following are true:

- ✓ Rollout completed successfully
- ✓ All Pods are running and ready
- ✓ Application health endpoint is healthy
- ✓ External endpoint returns expected response (200 OK)
- ✓ Error rate is within acceptable threshold
- ✓ Latency is within acceptable threshold
- ✓ No unexpected Pod restarts
- ✓ No critical alerts triggered
- ✓ Business / user scenario works as expected

If any of the above FAILS → the release is NOT successful.

! WHAT SHOULD TRIGGER ROLLBACK

Rollback immediately if any of the following occurs:

- ✗ Rollout fails or does not complete
- ✗ Pods crashloop or restart continuously
- ✗ Health endpoint returns errors
- ✗ External endpoint returns 5xx errors
- ✗ Error rate spikes above threshold
- ✗ Latency increases significantly
- ✗ Critical alerts fire
- ✗ Database connection failures
- ✗ Core user flows are broken
- ✗ Security incident detected

Never wait for the system to fully break.
Act fast, then investigate.

DEPLOYMENT VERIFICATION FLOW



1. DEPLOY
Release the new version to production.



2. OBSERVE
Collect metrics, logs, and system signals.



3. VERIFY
Validate against success criteria within the window.



4A. ACCEPT
Release is good. Continue monitoring.

OR



4B. ROLL BACK
Criteria failed. Roll back and investigate.



JUNIOR ENGINEER TIP

Always define how you will prove a deployment worked BEFORE you deploy.
If you cannot measure it, you cannot trust it.

16. PROJECT 15, PRODUCTION ROLLBACK

1 WHY ROLLBACK MUST BE PLANNED BEFORE DEPLOYMENT



- Failures happen.
- Fast rollback reduces downtime.
- Plan versions, data strategy, and verification.
- Define who decides and how.

2 IDENTIFY THE PREVIOUS STABLE VERSION



- Know what version is currently stable.
- Tag images and releases consistently.
- Keep release notes and version history.

```
kubectl rollout history
deploy/app -n app-prod
```

3 KUBERNETES ROLLOUT HISTORY



- Shows all revisions.
- Helps you pick the version to roll back to.
- Includes change-cause annotations.

```
kubectl rollout history
deploy/app -n app-prod
```

4 ROLL BACK DEPLOYMENT



- Roll back to a previous revision.
- Kubernetes updates Pods to the old version.

```
kubectl rollout undo
deploy/app -n app-prod
--to-revision=2
```

5 VERIFY ROLLBACK



- Check rollout status.
- Verify Pods, endpoints, and app health.
- Confirm metrics return to normal.

```
kubectl rollout status
deploy/app -n app-prod
```

6 RECORD FAILED RELEASE



- Capture what failed and why.
- Record timeline, impact, and actions.
- Share lessons learned.
- Update runbooks.

7. APPLICATION ROLLBACK VS DATABASE ROLLBACK



APPLICATION ROLLBACK

- Usually safe and fast.
- Roll back to a previous image version.
- No persistent data changes are reverted automatically.



DATABASE ROLLBACK

- Much harder and riskier.
- Data may have changed (updates, deletes, inserts).
- Rollback may cause data inconsistency or loss.



8. WHY DATABASE CHANGES COMPLICATE ROLLBACK

- Data is mutable and stateful.
- You cannot simply "roll back" a database to a previous state without backups.
- Schema changes may not be backward compatible.
- External systems may have already consumed the changed data.
- Often requires forward fix or data correction instead.

9. FORWARD FIX VS ROLLBACK

ROLLBACK



- Revert to the previous version.
- Best for quick recovery.
- Limited by database and external side effects.
- May only be a temporary fix.

FORWARD FIX



- Keep the current version.
- Fix the issue with a patch or config change.
- Often safer when data has already changed.
- Requires more investigation.



GUIDELINE

If the issue is code-related and data is safe → Rollback.
If data has changed or rollback is unsafe → Forward fix.

PRODUCTION ROLLBACK PROCESS FLOW



1. DETECT ISSUE

Monitor alerts, logs, metrics, or user reports.



2. ASSESS IMPACT

Determine severity and affected users.



3. CHECK HISTORY

Find the last known stable revision.



4. ROLLBACK

Undo to the selected stable revision.



5. VERIFY

Verify health, metrics, and user experience.



6. DOCUMENT

Record incident and update runbooks.

10. PRODUCTION FAILURE LAB: DEPLOY A BROKEN IMAGE AND RECOVER SAFELY



1. DEPLOY BROKEN IMAGE

Deploy an image with an error (e.g., crash on start).



2. OBSERVE FAILURE

Check Pods, logs, events, and metrics. Confirm user impact.



3. CHECK ROLLOUT HISTORY

Identify the last stable revision.



4. ROLLBACK

Execute rollback to previous stable revision.



5. VERIFY RECOVERY

Ensure Pods are healthy, metrics are normal, and users are not impacted.



6. DOCUMENT & LEARN

Record what happened, what fixed it, and how to prevent it.



LAB GOAL: Recover the application quickly and safely with minimal downtime and no data loss.



PRODUCTION AWARENESS: A COMPLETED PIPELINE IS NOT THE SAME AS A SUCCESSFUL PRODUCTION RELEASE.

Always be ready to roll back, and always learn from every failure.



JUNIOR ENGINEER TIP:

Always know how to get back to the last known good state before you go live.
A good engineer plans the rollback path before they need it.

17. PROJECT 16, SECRETS AND CI/CD CREDENTIALS



WHY PIPELINES NEED CREDENTIALS

- Pipelines interact with external systems.
- They need credentials to authenticate and perform actions.
- Examples: cloud, registry, kubernetes, databases, APIs.
- Without proper management, secrets become the biggest risk in your delivery system.

TYPES OF CREDENTIALS USED IN CI/CD



REPOSITORY SECRETS

- API tokens
- Webhook secrets
- Access tokens
- SSH keys



CLOUD CREDENTIALS

- AWS, Azure, GCP keys
- Service accounts
- Access keys
- IAM roles



REGISTRY CREDENTIALS

- Docker registry username
- Password or access token
- Pull / Push permissions



KUBERNETES CREDENTIALS

- kubeconfig files
- Service accounts
- Cluster access tokens



OTHER SYSTEM CREDENTIALS

- Database users
- Third-party API keys
- Monitoring credentials



NEVER HARDCODE SECRETS

- Never commit secrets to code.
- Never paste secrets in pipeline scripts.
- Never store secrets in plain text files.
- Never echo secrets in logs.

BAD (EXAMPLE)

```
export AWS_ACCESS_KEY=AKIA...
password: "mypassword123"
docker login -u user -p password
```



GOOD (USE SECRET STORES)

- Use built-in secret stores.
- Inject secrets at runtime.
- Keep secrets encrypted at rest.
- Limit access to only what is needed.



SECRET MASKING

- Secrets must be masked in logs.
- Use platform secret masking features.
- Always verify masking works.
- If a secret appears in logs, treat it as a breach.

Log Output Example

Token: *****



LEAST PRIVILEGE

- Use the minimum permissions required.
- Separate credentials by purpose.
- One credential should not be able to impact everything.
- Review permissions regularly.

Principle: Grant the least, for the shortest time, for the specific task.



SHORT-LIVED CREDENTIALS

- Short-lived keys reduce risk.
- Use tokens that expire automatically.
- Prefer temporary credentials (STS, OIDC, Workload Identity).
- Avoid long-lived static access keys.

Short-lived = Lower risk window if a secret is exposed.



WORKLOAD IDENTITY & OIDC INTRODUCTION

- OIDC allows your CI/CD system to get temporary credentials securely.
- No long-lived static keys required.
- Federated identity establishes trust between CI/CD and cloud provider.
- Examples: GitHub Actions OIDC, GitLab OIDC, Azure Federated Credentials, AWS STS with OIDC.

Modern pipelines should prefer
OIDC + Workload Identity.



SECRET ROTATION

- Rotate secrets regularly.
- Immediately rotate after any suspicion of exposure.
- Automate rotation where possible.
- Document rotation process.

Rotate → Reduce → Respond → Harden



QUICK SECURITY CHECKLIST

- ☒ Secrets stored in secure stores
- ☒ No secrets in code or pipeline files
- ☒ Secrets masked in logs
- ☒ Least privilege applied
- ☒ Short-lived credentials used
- ☒ OIDC / Workload Identity enabled
- ☒ Regular secret rotation
- ☒ Access reviewed regularly



SECURITY REMINDER

CI/CD SYSTEMS ARE PRIVILEGED INFRASTRUCTURE
AND SHOULD BE TREATED AS HIGH-VALUE SECURITY TARGETS.



18. PROJECT 17, PIPELINE PERFORMANCE AND RELIABILITY



WHY SLOW PIPELINES HURT DELIVERY

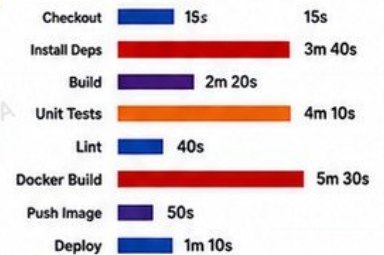
- Developers wait longer for feedback.
- Slower releases and more lead time.
- Context switching reduces productivity.
- Blocks other teams and environments.
- Increases risk and delays business value.
- Fast, reliable pipelines build trust in delivery.



IDENTIFY SLOW STAGES

- Measure each stage duration.
- Use pipeline timing reports.
- Look for long-running steps.
- Check for unnecessary work.
- Analyze historical trends.
- Focus on the top 20% slowest stages.

EXAMPLE: STAGE DURATION



KEY STRATEGIES TO IMPROVE PIPELINE PERFORMANCE



DEPENDENCY CACHING

- Cache package manager directories.
- Reuse dependencies between runs.
- Greatly reduces install time.

Example:

`~/.m2, ~/.npm, pip cache`



DOCKER LAYER CACHING

- Cache Docker layers between builds.
- Avoid rebuilding unchanged layers.
- Speeds up image builds.

Example:

`--cache-from / --cache-to`



PARALLEL JOBS

- Run independent jobs in parallel.
- Tests, linting, scanning can run together.
- Reduces total pipeline time significantly.

Example:

`matrix builds`



CONDITIONAL EXECUTION

- Run jobs only when needed.
- Skip docs, tests, or builds for minor changes.
- Use rules / filters / paths.

Example:

`rules:if, only, except`



REUSABLE PIPELINE COMPONENTS

- Extract common steps into templates.
- Use reusable workflows, includes, or modules.
- Ensures consistency and saves time.

Example:

`template / reusable job`



RETRY STRATEGY

- Retry transient failures automatically.
- Helps with flaky networks and external services.
- Don't hide real failures.

Example:

`retry: 2`



TIMEOUTS

- Set timeouts for jobs and steps.
- Prevent stuck jobs from blocking pipeline.
- Fail fast, fix fast.

Example:

`timeout: 10m`



FLAKY TESTS

- Identify and stabilize flaky tests.
- Isolate external dependencies.
- Use retries with caution.
- Track flake rate.

Tip:

`Fix the test, not hide it.`



RUNNER CAPACITY

- Ensure enough runners for parallel jobs.
- Monitor queue times.
- Scale runners based on demand.
- Right size = fast pipelines.

Monitor:

`Queue time & utilization`

PIPELINE PERFORMANCE OPTIMIZATION FLOW



1. MEASURE

Measure and collect pipeline metrics.



2. IDENTIFY

Find the slowest stages.



3. OPTIMIZE

Apply the right strategies.



4. VALIDATE

Run again and verify improvement.



5. REPEAT

Continuously improve over time.



METRICS THAT MATTER

- Pipeline duration
- Stage duration
- Queue time
- Success rate
- Deployment frequency
- Lead time for changes
- MTTR (for failures)



PERFORMANCE TIP:

OPTIMIZE BOTTLENECKS INSTEAD OF BLINDLY ADDING PARALLEL JOBS.
MEASURE, UNDERSTAND, AND IMPROVE WHAT REALLY SLOWS YOU DOWN.



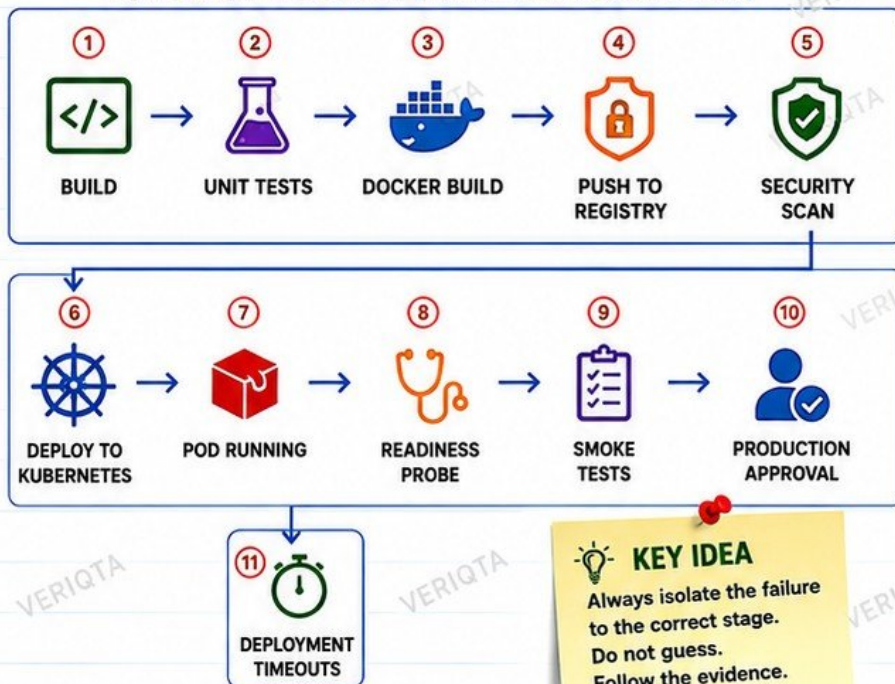
19. PROJECT 18, TROUBLESHOOT A BROKEN PRODUCTION PIPELINE

HOW TO FIND, UNDERSTAND, AND FIX FAILURES AT EVERY STAGE

WE WILL INTRODUCE FAILURES AT MULTIPLE STAGES

- 1 BUILD FAILS
- 2 UNIT TEST FAILS
- 3 DOCKER BUILD FAILS
- 4 REGISTRY AUTHENTICATION FAILS
- 5 VULNERABILITY GATE BLOCKS RELEASE
- 6 KUBERNETES DEPLOYMENT FAILS
- 7 POD ENTERS CRASHLOOPBACKOFF
- 8 READINESS PROBE FAILS
- 9 SMOKE TEST FAILS
- 10 PRODUCTION APPROVAL IS BLOCKED
- 11 DEPLOYMENT TIMES OUT

EXAMPLE PIPELINE FLOW (WHERE THINGS CAN BREAK)



REMEMBER

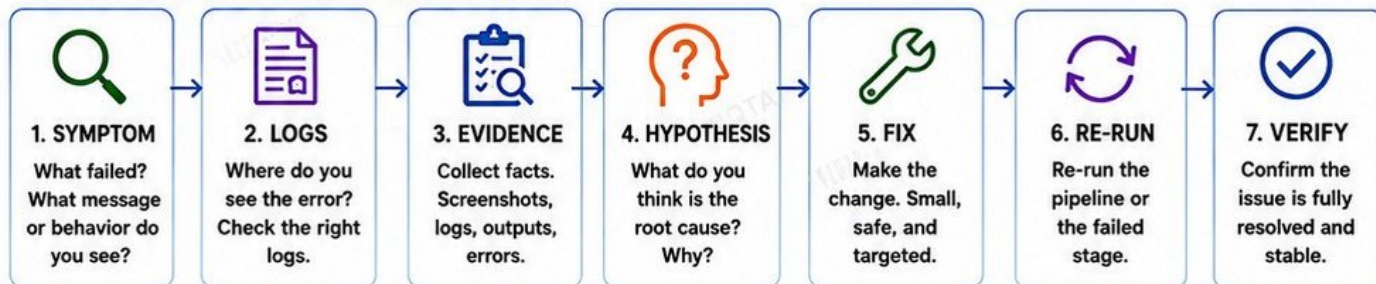
A pipeline can fail for many reasons. Your job is to identify **WHICH STAGE OWNS THE FAILURE**.



KEY IDEA

Always isolate the failure to the correct stage. Do not guess. Follow the evidence.

TROUBLESHOOTING FLOW (USE THIS EVERY TIME)



COMMON DEBUGGING COMMANDS (EXAMPLES)

<code>kubectl get pods</code>	Check pod status in the namespace.
<code>kubectl describe pod <pod-name></code>	See events and probe failures.
<code>kubectl logs <pod-name></code>	View application logs.
<code>kubectl logs <pod-name> -c <container></code>	View logs of a specific container.
<code>kubectl get events --sort-by=.lastTimestamp</code>	See recent Kubernetes events.
<code>kubectl rollout status deployment/<name></code>	Check rollout progress and failures.
<code>kubectl get svc</code>	Verify services and cluster IPs.
<code>kubectl get ingress</code>	Check ingress configuration.

BEGINNER MISTAKES TO AVOID

- ✗ Ignoring the actual error message
- ✗ Looking at the wrong stage logs
- ✗ Assuming the problem is in the application
- ✗ Changing too many things at once
- ✗ Not re-running and verifying
- ✗ Fixing symptoms, not root causes



GOAL

Become comfortable reading errors, logs, and system signals. A good engineer knows how to investigate, not just how to deploy.



REAL WORLD LESSON

Most production issues are not caused by code. They are caused by configuration, dependencies, environment, or assumptions.



VeriQta
YouTube



veriQta
GitHub



veriQta
LinkedIn



veriQta
X



veriQta
Instagram

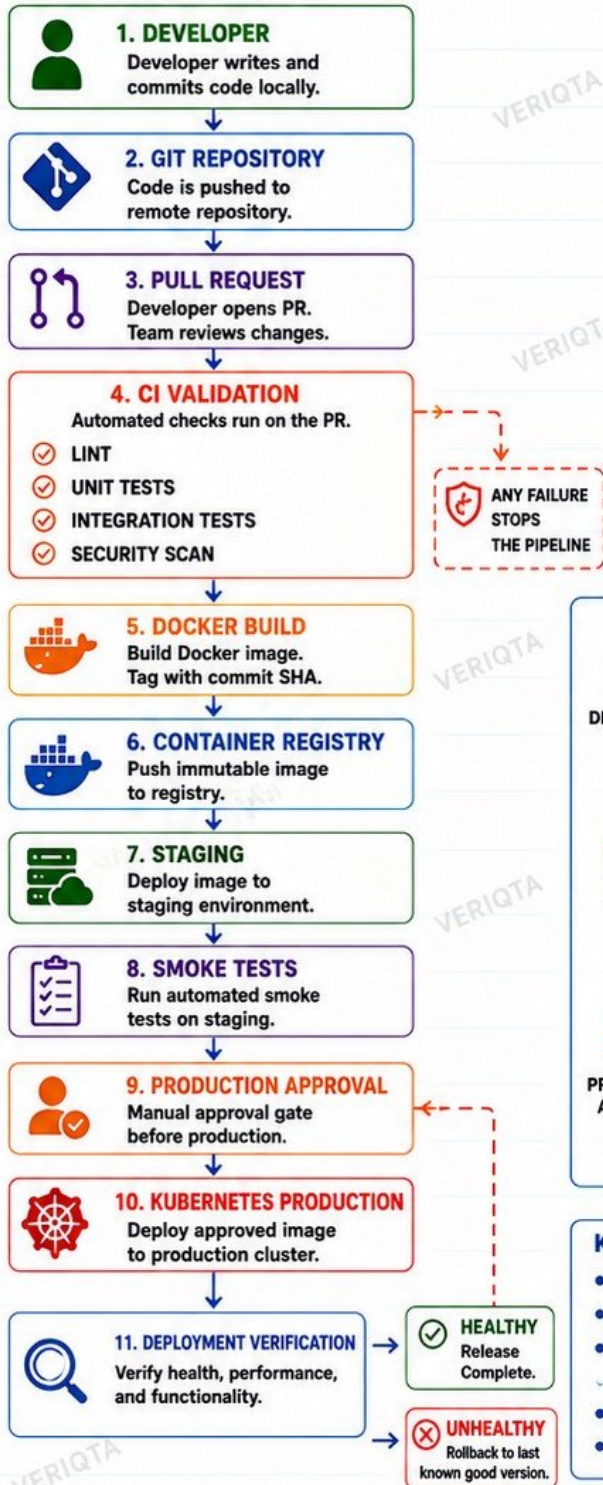


veriQta
Telegram

20. FINAL PROJECT, BUILD THE COMPLETE PRODUCTION CI/CD SYSTEM

BUILD, TEST, SECURE, AND DEPLOY A REAL-WORLD CI/CD PIPELINE FROM END TO END

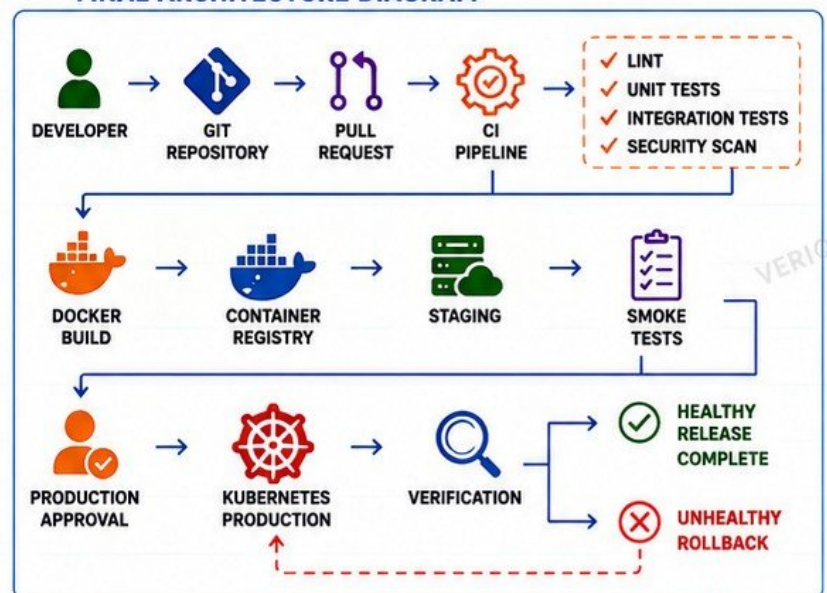
COMPLETE WORKFLOW



FINAL PROJECT REQUIREMENTS

- ✓ GIT-BASED WORKFLOW
- ✓ PULL REQUEST VALIDATION
- ✓ AUTOMATED TESTING
- ✓ SECURITY SCANNING
- ✓ IMMUTABLE DOCKER IMAGE
- ✓ CONTAINER REGISTRY
- ✓ STAGING DEPLOYMENT
- ✓ SMOKE TESTING
- ✓ PRODUCTION APPROVAL
- ✓ KUBERNETES PRODUCTION DEPLOYMENT
- ✓ HEALTH VERIFICATION
- ✓ ROLLBACK PROCEDURE
- ✓ SECRETS HANDLED SECURELY
- ✓ PIPELINE NOTIFICATIONS
- ✓ DEPLOYMENT TRACEABILITY
- ✓ TROUBLESHOOTING DOCUMENTATION

FINAL ARCHITECTURE DIAGRAM



KEY PRINCIPLES

- BUILD ONCE, DEPLOY MANY TIMES
- EVERY CHANGE IS TESTED AND VERIFIED
- AUTOMATE EVERYTHING YOU CAN
- PROTECT PRODUCTION WITH APPROVALS AND CHECKS
- OBSERVE AND VERIFY BEFORE DECLARING SUCCESS
- ROLLBACK IS PART OF THE PLAN, NOT A LAST RESORT

DEVOPS CONNECTION

Follow one code change through the entire system. Understand how each stage adds value and safety.